

Exploratory Data Analysis (EDA) & Feature Engineering

1. Objective

- Conduct Exploratory Data Analysis (EDA) on preprocessed beauty & wellness market data.
- Identify customer patterns, market trends, and actionable insights.
- Design engineered features that improve predictive modeling and business decision-making.

2. Dataset Overview

- **Source:** Sales, CRM, reviews, POS, or scraped data.
- **Entities:** Customers, Transactions, Products, Reviews, Stores/Regions, Campaigns, Dates.
- **Key Fields:** customer_id, transaction_id, product_id, category, price, quantity, rating, review_text, promo_flag.
- **Preprocessing Summary:** Missing values handled, duplicates removed, data types corrected, derived fields created (e.g., total_amount = price × quantity).

3. EDA Workflow

Step-by-Step Plan

1. **Data Quality Checks:** Missing values, duplicates, data types.
2. **Univariate Analysis:** Histograms, boxplots, KDE for numeric features.
3. **Bivariate Analysis:** Scatter plots, grouped boxplots, correlation heatmaps.
4. **Multivariate Analysis:** PCA, clustering, pairplots.
5. **Time Series Analysis:** Seasonal trends, rolling averages, campaign impact.
6. **Text Analysis:** Sentiment scoring from reviews, word frequency.
7. **Outlier Detection:** Z-score, IQR method.

Visualization Tools

- **Matplotlib & Seaborn:** Histograms, bar charts, heatmaps, violin plots.
- **SciPy/Statsmodels:** Statistical tests (t-test, ANOVA, chi-square).
- **Wordcloud/Plotly:** Text and customer journey visualizations.

4. Statistical Methods

- **Descriptive Stats:** Mean, median, variance, percentiles.
- **Tests:** t-test, ANOVA, chi-square, correlation (Pearson/Spearman).
- **Effect Size:** Cohen's d, Cramér's V.
- **Multiple Testing:** Bonferroni or Benjamini-Hochberg corrections.

5. Feature Engineering Plan

Candidate Features

- **Customer Behavior:** Recency, Frequency, Monetary (RFM), average basket size, churn flag.

- **Product Features:** Popularity index, return rate, price elasticity proxy.
- **Engagement:** Channel preference, promo responsiveness.
- **Temporal:** Day of week, month, seasonal index.
- **Text-Derived:** Sentiment score, review length, keyword flags.
- **Interaction Features:** Price × Frequency, Category × Region, Segment × Promo_flag.

Transformations

- Normalization (Min-Max, StandardScaler).
- Encoding (One-Hot, Label, Target encoding).
- Log transformations for skewed values.
- PCA/SVD for dimensionality reduction.

6. Hypothetical Scenario

Example:

- Raw data: visit frequency + average spend.
- Engineered feature: **Customer Value Score = frequency × spend.**
- Insight: Identify high-value customers → target with premium offers → increase revenue.

7. Model Readiness

- Check multicollinearity (VIF).
- Feature importance via RandomForest/Lasso.
- Cross-validation with engineered features vs baseline.
- Track metrics: AUC, RMSE, conversion uplift, retention rate.

8. Workload Estimate (30–35 Hours)

- 4 hrs → Data quality checks.
- 8 hrs → EDA plots & analysis.
- 6 hrs → Time series & sentiment analysis.
- 8 hrs → Feature engineering design.
- 5–6 hrs → Documentation & business mapping.

9. DOC File Structure

Title Page Executive Summary 1. Introduction 2. Data Overview 3. EDA Workflow 4. Statistical Methods 5. Feature Engineering Strategy 6. Business Insights Mapping 7. Model Readiness & Validation 8. Implementation Plan & Timeline 9. Appendix (Python code snippets, plots, glossary)

10. Example Python Snippets

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Load data
```

```
df = pd.read_csv('beauty_data.csv')
```

```
# Univariate plot
```

```
sns.histplot(df['total_amount'], bins=50, kde=True)
```

```
plt.title('Distribution of Total Amount')
```

```
plt.show()
```

```
# Correlation heatmap
```

```
corr = df[['price', 'quantity', 'total_amount', 'rating']].corr()
```

```
sns.heatmap(corr, annot=True, cmap='coolwarm')
```

```
plt.title('Correlation Matrix')
```

```
plt.show()
```

```
# Feature engineering: Customer Value Score
```

```
cust = df.groupby('customer_id').agg({'transaction_id': 'nunique', 'total_amount': 'sum'})
```

```
cust['avg_spend'] = cust['total_amount'] / cust['transaction_id']
```

```
cust['customer_value_score'] = cust['transaction_id'] * cust['avg_spend']
```